[Lecture Notebook] Analysis Technique Summary

Summary of Chapter6 to Chapter20

Chad (Chungil Chae)

Tue, 27 May 2025

Multiple Linear Regression (Ch.6)

Data

```
# Sample housing dataset
data <- data.frame(
  Size = c(1000, 1500, 2000, 2500, 3000, 1800, 1600, 2400, 2200, 2800),
  Bedrooms = c(2, 3, 3, 4, 4, 3, 2, 4, 3, 4),
  Price = c(200000, 250000, 270000, 310000, 360000, 260000, 230000, 300000, 280000, 340000)
)
```

Train-Test Split

```
set.seed(123)  # For reproducibility
sample_index <- sample(1:nrow(data), 0.7 * nrow(data))
train_data <- data[sample_index, ]
test_data <- data[-sample_index, ]
```

Model Training

```
# Fit the regression model
model <- lm(Price ~ Size + Bedrooms, data = train_data)

# Model summary
summary(model)
```

```
Call:
lm(formula = Price ~ Size + Bedrooms, data = train_data)

Residuals:
      3      10       2       8       6       9       1
-1739.1  8415.1  6451.6 -9032.3  -462.8 -3015.4  -617.1

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) 114754.56   14958.83   7.671  0.00155 **
Size            56.38      12.04   4.683  0.00943 **
Bedrooms     14740.53   10364.62   1.422  0.22804
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 7189 on 4 degrees of freedom
Multiple R-squared:  0.9817,    Adjusted R-squared:  0.9725
F-statistic: 107.2 on 2 and 4 DF,  p-value: 0.0003356
```

Prediction

```
# Predict on test data
predictions <- predict(model, newdata = test_data)
predictions
```

```
       4        5        7
314670.4 342861.2 234446.0
```

Evaluation Metrics

```
#install.packages("Metrics")
library(Metrics)

actuals <- test_data$Price

mae <- mean(abs(actuals - predictions))

rmse <- sqrt(mean((actuals - predictions)^2))

SSE <- sum((actuals - predictions)^2)
```



```
SST <- sum((actuals - mean(actuals))^2)
rsq <- 1 - SSE/SST

cat("MAE:", round(mae, 2), "\n")
```

```
MAE: 8751.75
```

```
cat("RMSE:", round(rmse, 2), "\n")
```

```
RMSE: 10572.29
```

```
cat("R-squared:", round(rsq, 4), "\n")
```

```
R-squared: 0.961
```

k-Nearest Neighbors (kNN) (Ch.7)

Data

```
# Sample height-weight-gender dataset
data <- data.frame(
  Height = c(5.1, 6.0, 5.5, 6.2, 5.8, 5.3, 5.9, 6.1, 5.4, 5.7),
  Weight = c(120, 180, 150, 200, 160, 130, 170, 190, 140, 155),
  Gender = as.factor(c("F", "M", "F", "M", "F", "F", "M", "M", "F", "F"))
)
```

Train-Test Split

```
set.seed(123)
sample_index <- sample(1:nrow(data), 0.7 * nrow(data))
train_data <- data[sample_index, ]
test_data <- data[-sample_index, ]
```

Feature Scaling

```r
# Normalize the features
normalize <- function(x) { (x - min(x)) / (max(x) - min(x)) }

train_scaled <- as.data.frame(lapply(train_data[, 1:2], normalize))
test_scaled <- as.data.frame(lapply(test_data[, 1:2], normalize))

# Labels
train_labels <- train_data$Gender
test_labels <- test_data$Gender
```

39 **kNN Model Training and Prediction**

40 **Data**

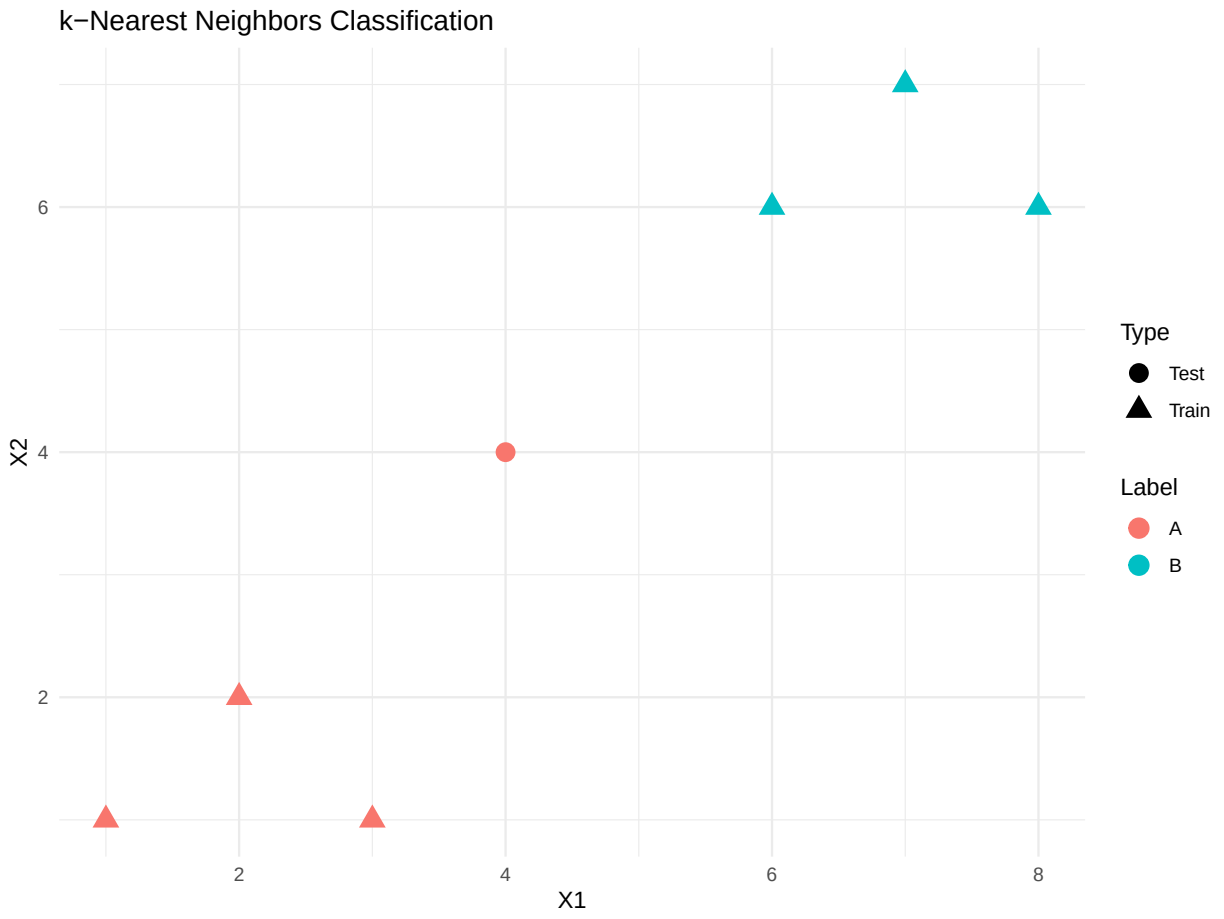
```r
# install.packages("ggplot2")
library(ggplot2)
library(class)



set.seed(123)

# Training data
train_data <- data.frame(
  X1 = c(1, 2, 3, 6, 7, 8),
  X2 = c(1, 2, 1, 6, 7, 6),
  Label = as.factor(c("A", "A", "A", "B", "B", "B"))
)

# Test point
test_data <- data.frame(X1 = c(4), X2 = c(4))
```

41 **kNN Prediction**

```r
# Scale if needed (not necessary in this small example)
train_matrix <- train_data[, 1:2]
test_matrix <- test_data

predicted <- class::knn(train = train_matrix, test = test_matrix, cl = train_data$Label, k = 3)
```


42 **Visualization**

```
# Add prediction result to test data
test_data$Label <- predicted

# Combine for plotting
combined <- rbind(
  data.frame(train_data, Type = "Train"),
  data.frame(test_data, Type = "Test")
)

# Plot
ggplot(combined, aes(x = X1, y = X2, color = Label, shape = Type)) +
  geom_point(size = 4) +
  labs(title = "k-Nearest Neighbors Classification") +
  scale_shape_manual(values = c(16, 17)) +
  theme_minimal()
```



43

The Naive Bayes Classifier

Data

```r
# Sample weather dataset
data <- data.frame(
  Outlook = c("Sunny", "Sunny", "Overcast", "Rain", "Rain", "Rain", "Overcast", "Sunny", "Sunny",
  Temp = c("Hot", "Hot", "Hot", "Mild", "Cool", "Cool", "Cool", "Mild", "Cool", "Mild"),
  Play = as.factor(c("No", "No", "Yes", "Yes", "Yes", "No", "Yes", "No", "Yes", "Yes"))
)
```

Train-test split

```r
set.seed(123)
sample_index <- sample(1:nrow(data), 0.7 * nrow(data))
train_data <- data[sample_index, ]
test_data <- data[-sample_index, ]
```

Naive Bayes model training

```r
library(e1071)

# Train model
model <- naiveBayes(Play ~ ., data = train_data)
```

Prediction

```r
# Predict on test data
predicted <- predict(model, newdata = test_data)
```

Evaluation (accuracy & confusion matrix)

```r
# Accuracy
accuracy <- mean(predicted == test_data$Play)

# Confusion matrix
conf_matrix <- table(Predicted = predicted, Actual = test_data$Play)

# Output
cat("Accuracy:", round(accuracy, 2), "\n")
```

```
Accuracy: 1
```

```r
print(conf_matrix)
```

```
         Actual
Predicted No Yes
      No   0   0
      Yes  0   3
```

Classification and Regression Trees

Data

```r
# Customer data for purchase decision
data <- data.frame(
  Age = c(25, 45, 35, 33, 50, 29, 40, 38, 48, 55),
  Income = as.factor(c("High", "Medium", "Low", "Medium", "High", "Low", "Medium", "High", "Low",
  Buy = as.factor(c("No", "Yes", "Yes", "Yes", "No", "Yes", "Yes", "No", "Yes", "Yes"))
)
```

Train-test split

```r
set.seed(123)
sample_index <- sample(1:nrow(data), 0.7 * nrow(data))
train_data <- data[sample_index, ]
test_data <- data[-sample_index, ]
```

Tree model training

```
library(rpart)

# Train decision tree model
tree_model <- rpart(
  Buy ~ Age + Income,
  data = train_data,
  method = "class",
  control = rpart.control(minsplit = 1, cp = 0)
)
```

Prediction

```
# Predict on test data
predicted <- predict(tree_model, newdata = test_data, type = "class")
```

Evaluation (accuracy and confusion matrix)

```
# Accuracy
accuracy <- mean(predicted == test_data$Buy)

# Confusion matrix
conf_matrix <- table(Predicted = predicted, Actual = test_data$Buy)

# Output
cat("Accuracy:", round(accuracy, 2), "\n")
```

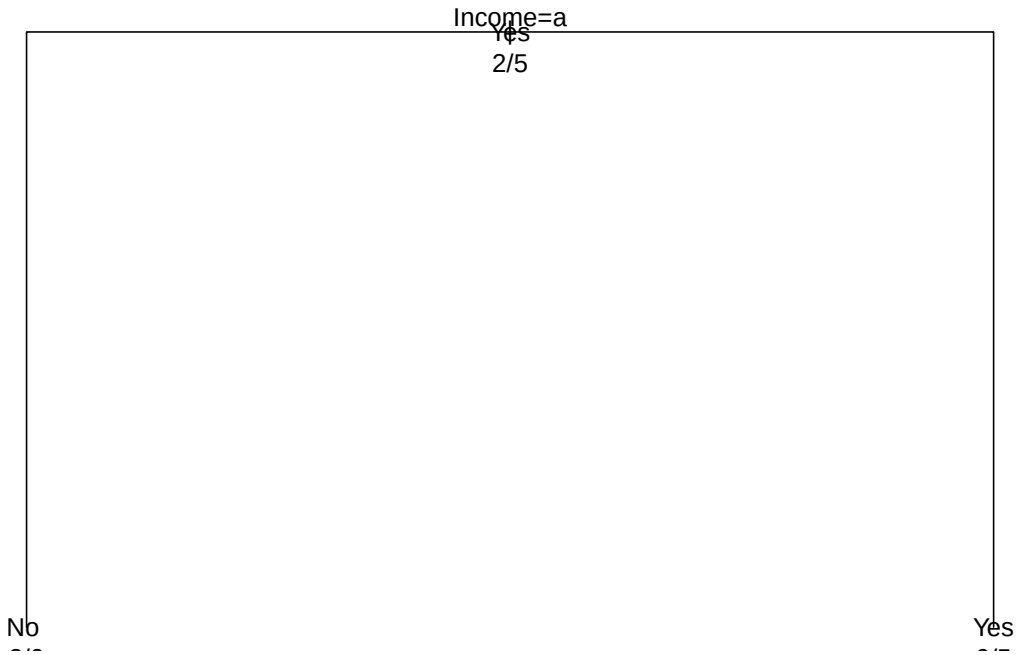
```
Accuracy: 1
```

```
print(conf_matrix)
```

```
         Actual
Predicted No Yes
      No   1   0
      Yes  0   2
```


66 **Tree visualization**

```
# Optional tree visualization
plot(tree_model)
text(tree_model, use.n = TRUE, all = TRUE)
```



67

```
table(train_data$Buy)
```

68

69 No Yes
70 2 5

Logistic Regression (Ch.10)

Data

```r
# Binary classification dataset (Purchased: 0 = No, 1 = Yes)
data <- data.frame(
  Age = c(22, 25, 47, 52, 46, 56, 23, 39, 48, 33),
  Salary = c(30000, 35000, 80000, 90000, 70000, 100000, 32000, 61000, 85000, 40000),
  Purchased = as.factor(c(0, 0, 1, 1, 1, 1, 0, 1, 1, 0))
)
```

Train-test split

```r
set.seed(123)
sample_index <- sample(1:nrow(data), 0.7 * nrow(data))
train_data <- data[sample_index, ]
test_data <- data[-sample_index, ]
```

Logistic model training

```r
# Logistic regression model
model <- glm(Purchased ~ Age + Salary, data = train_data, family = binomial)
summary(model)
```

```

Call:
glm(formula = Purchased ~ Age + Salary, family = binomial, data = train_data)

Coefficients:
              Estimate Std. Error z value Pr(>|z|)
(Intercept) -7.597e+01  3.180e+05       0        1
Age         -1.717e+00  2.262e+04       0        1
Salary       2.717e-03  1.092e+01       0        1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 9.5607e+00  on 6  degrees of freedom
Residual deviance: 4.1857e-10  on 4  degrees of freedom
AIC: 6
```



```
Number of Fisher Scoring iterations: 24
```

Prediction and evaluation (accuracy, confusion matrix)

```
# Predict probabilities
predicted_probs <- predict(model, newdata = test_data, type = "response")

# Classify based on 0.5 threshold
predicted_classes <- ifelse(predicted_probs > 0.5, 1, 0)
```

Evaluation

```
# Accuracy
actual <- as.numeric(as.character(test_data$Purchased))
accuracy <- mean(predicted_classes == actual)

# Confusion Matrix
conf_matrix <- table(Predicted = predicted_classes, Actual = actual)

# Output
cat("Accuracy:", round(accuracy, 2), "\n")
```

```
Accuracy: 1
```

```
print(conf_matrix)
```

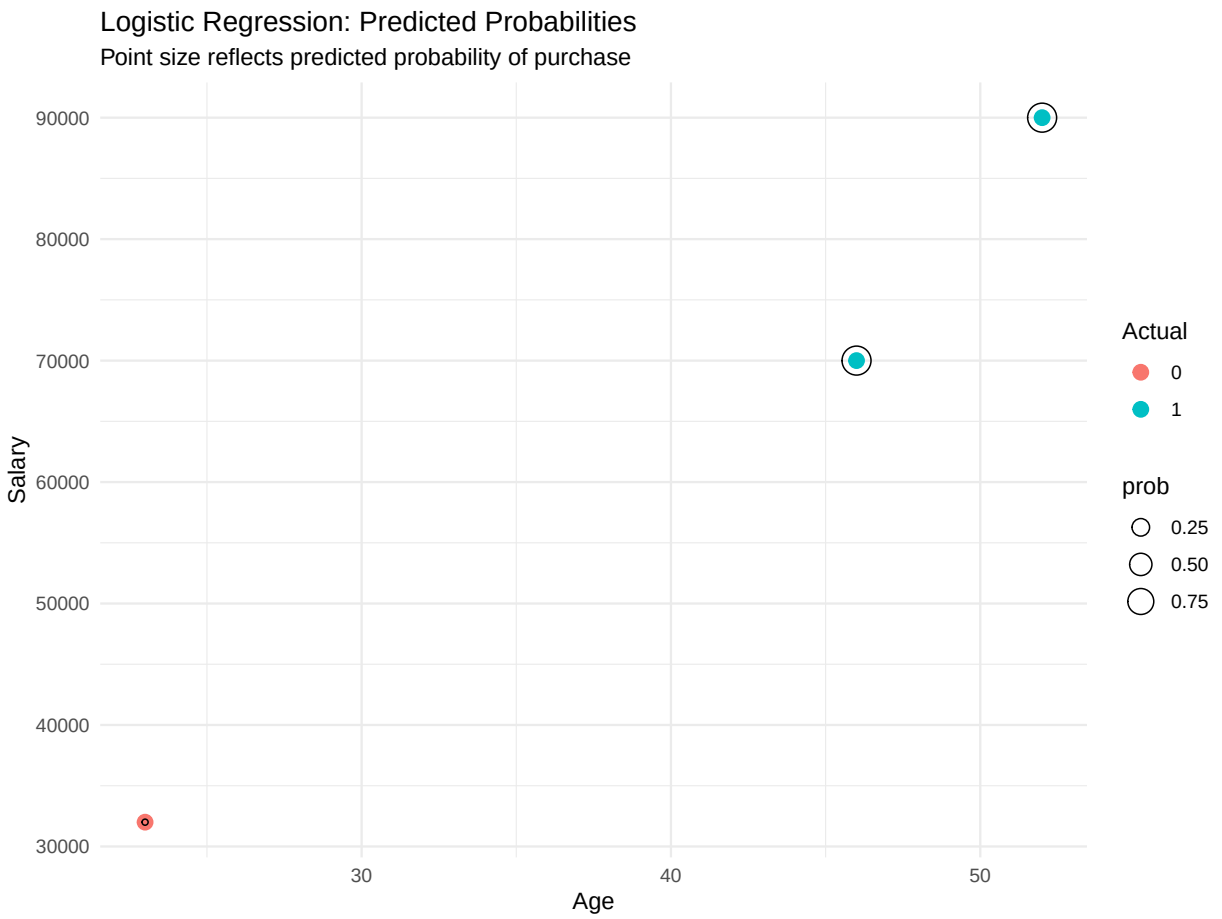
```
         Actual
Predicted 0 1
        0 1 0
        1 0 2
```

Visualization of decision boundary and probabilities

```
library(ggplot2)

# Add predicted probabilities to test data
test_data$prob <- predicted_probs
```



```r
test_data$predicted <- as.factor(predicted_classes)

# Plot Salary vs Age with predicted probability
ggplot(test_data, aes(x = Age, y = Salary)) +
  geom_point(aes(color = Purchased), size = 3) +
  geom_point(aes(size = prob), color = "black", shape = 1) +
  labs(title = "Logistic Regression: Predicted Probabilities",
       subtitle = "Point size reflects predicted probability of purchase",
       color = "Actual") +
  theme_minimal()
```



100

Neural Nets (Ch.11)

Data

```
# Simulated data: test scores based on study and sleep hours
data <- data.frame(
  StudyHours = c(1, 2, 3, 4, 5, 6, 7, 8, 9, 10),
  SleepHours = c(7, 6.5, 6, 5.5, 5, 4.5, 4, 3.5, 3, 2.5),
  Score = c(50, 55, 60, 65, 70, 72, 74, 76, 78, 80)
)
```

Feature scaling

```
# Normalize features to 0-1 range
normalize <- function(x) (x - min(x)) / (max(x) - min(x))
data_scaled <- as.data.frame(lapply(data, normalize))
```

Train-test split

```
set.seed(123)
sample_index <- sample(1:nrow(data_scaled), 0.7 * nrow(data_scaled))
train_data <- data_scaled[sample_index, ]
test_data <- data_scaled[-sample_index, ]
```

Model training

```
library(nnet)

# Train neural network (1 hidden layer with 3 neurons)
nn_model <- nnet(Score ~ StudyHours + SleepHours,
                 data = train_data,
                 size = 3,
                 linout = TRUE,
                 maxit = 500,
                 trace = FALSE)
```

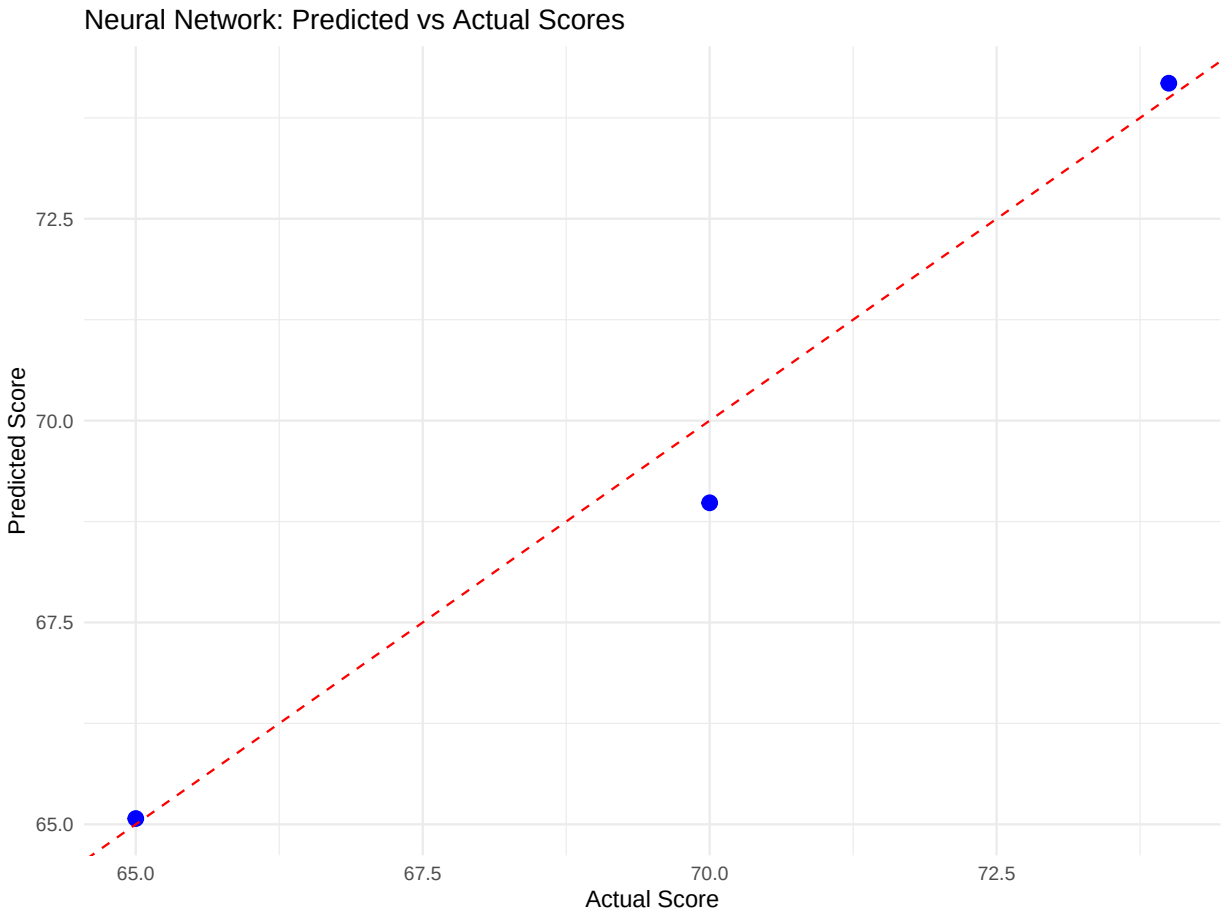

106 **Prediction and evaluation**

```
# Predict on test data
predicted <- predict(nn_model, newdata = test_data)

# Denormalize (for interpretability)
denormalize <- function(x, orig) x * (max(orig) - min(orig)) + min(orig)
actual_score <- denormalize(test_data$Score, data$Score)
pred_score <- denormalize(predicted, data$Score)

# RMSE
rmse <- sqrt(mean((pred_score - actual_score)^2))
cat("RMSE:", round(rmse, 2), "\n")
```

107 RMSE: 0.6

108 **Visualization of predicted vs actual output**

```
library(ggplot2)

df_plot <- data.frame(
  Actual = actual_score,
  Predicted = pred_score
)

ggplot(df_plot, aes(x = Actual, y = Predicted)) +
  geom_point(color = "blue", size = 3) +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed", color = "red") +
  labs(title = "Neural Network: Predicted vs Actual Scores",
       x = "Actual Score", y = "Predicted Score") +
  theme_minimal()
```

Discriminant Analysis (Ch.12)

Data

```
# Students classified into classes based on scores
data <- data.frame(
  Math = c(80, 60, 70, 90, 50, 85, 45, 78, 66, 74),
  Science = c(85, 55, 65, 95, 45, 90, 40, 82, 61, 72),
  Class = as.factor(c("A", "B", "B", "A", "B", "A", "B", "A", "B", "A"))
)
```

Train-test split


```
set.seed(123)
sample_index <- sample(1:nrow(data), 0.7 * nrow(data))
train_data <- data[sample_index, ]
test_data <- data[-sample_index, ]
```

Model training with LDA

```
library(MASS)

lda_model <- lda(Class ~ Math + Science, data = train_data)
```

Prediction and evaluation

```
# Predict on test data
lda_pred <- predict(lda_model, newdata = test_data)

# View predicted classes
predicted_classes <- lda_pred$class
```

Evaluation

```
# Accuracy
actual_classes <- test_data$Class
accuracy <- mean(predicted_classes == actual_classes)

# Confusion matrix
conf_matrix <- table(Predicted = predicted_classes, Actual = actual_classes)

# Output
cat("Accuracy:", round(accuracy, 2), "\n")
```

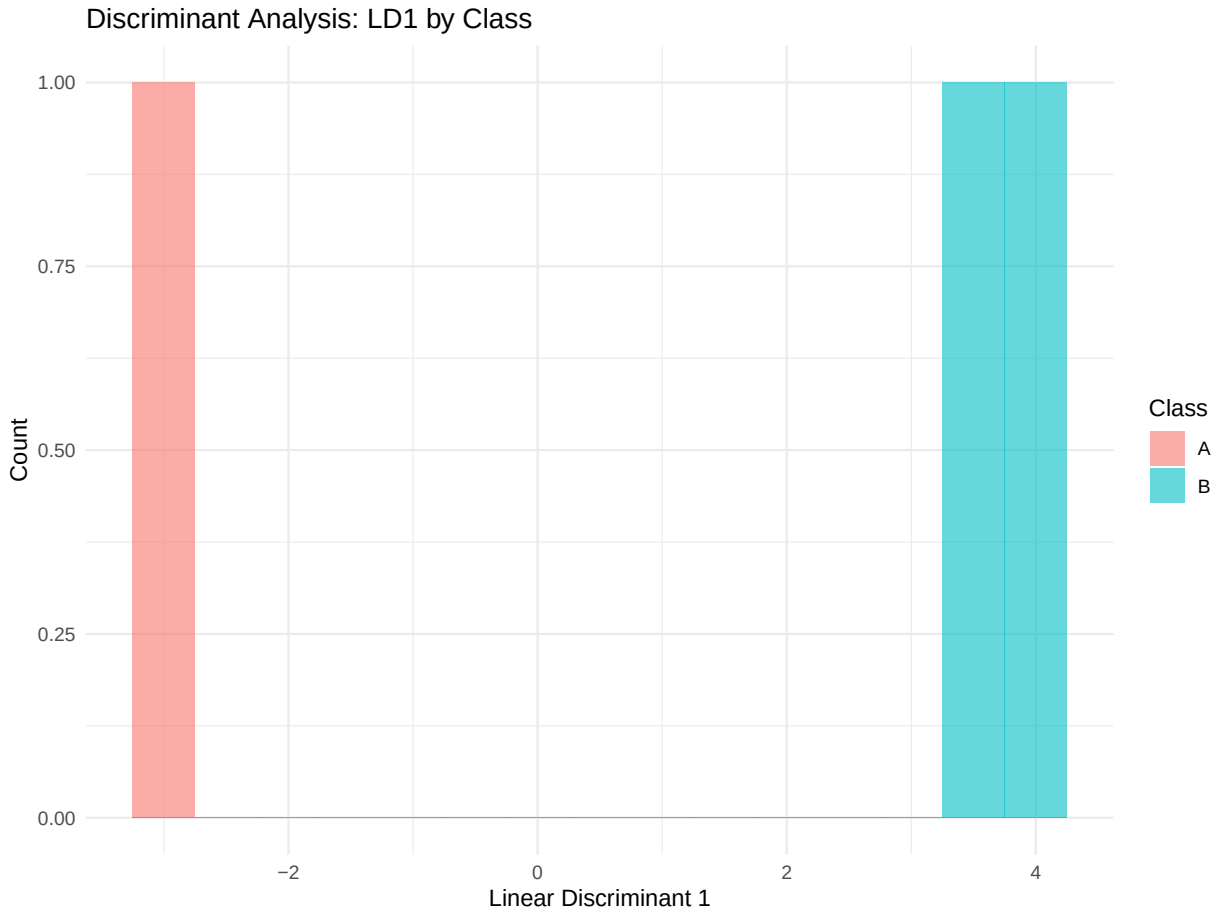
```
Accuracy: 1
```

```
print(conf_matrix)
```

```
          Actual
Predicted A B
        A 1 0
        B 0 2
```


Visualization of discriminant scores

```
library(ggplot2)

# Combine LDA output with test data for plotting
plot_data <- data.frame(
  LD1 = lda_pred$x[,1],
  Class = actual_classes,
  Predicted = predicted_classes
)

ggplot(plot_data, aes(x = LD1, fill = Class)) +
  geom_histogram(binwidth = 0.5, position = "identity", alpha = 0.6) +
  labs(title = "Discriminant Analysis: LD1 by Class",
       x = "Linear Discriminant 1", y = "Count") +
  theme_minimal()
```



Combining Methods: Ensembles and Uplift Modeling (Ch.13)

Ensemble Learning using a Random Forest

Data

```
# Binary classification problem
data <- data.frame(
  Feature1 = c(2, 4, 6, 8, 10, 1, 3, 5, 7, 9),
  Feature2 = c(1, 3, 5, 7, 9, 2, 4, 6, 8, 10),
  Label = as.factor(c(0, 0, 1, 1, 1, 0, 0, 1, 1, 1))
)
```

Train-Test Split

```
set.seed(123)
sample_index <- sample(1:nrow(data), 0.7 * nrow(data))
train_data <- data[sample_index, ]
test_data <- data[-sample_index, ]
```

Random Forest Ensemble Model

```
library(randomForest)

# Train random forest
rf_model <- randomForest(Label ~ ., data = train_data, ntree = 100)

# Predict on test data
predicted <- predict(rf_model, newdata = test_data)

# Accuracy
accuracy <- mean(predicted == test_data$Label)
cat("Random Forest Accuracy:", round(accuracy, 2), "\n")
```

```
Random Forest Accuracy: 1
```


129 **A basic Uplift Modeling case using the tools4uplift package**

130 **Data**

```
#install.packages("grf")
library(grf)
set.seed(123)

n <- 500
X <- matrix(rnorm(n * 3), ncol = 3)  # Covariates
colnames(X) <- c("X1", "X2", "X3")

W <- rbinom(n, 1, 0.5)  # Random treatment assignment

# Outcome depends on features and treatment interaction (heterogeneous effect)
Y <- 0.5 + 0.5 * X[,1] + 0.5 * X[,2] + W * (1 + 0.5 * X[,1]) + rnorm(n)
```

131 **Train Causal Forset Model**

```
# Train causal forest to estimate conditional average treatment effect (CATE)
cf_model <- causal_forest(X, Y, W)

### Predict uplift scores
# Predict uplift (CATE) for each individual
cate_pred <- predict(cf_model)

# View first few predicted uplift scores
head(cate_pred$predictions)
```

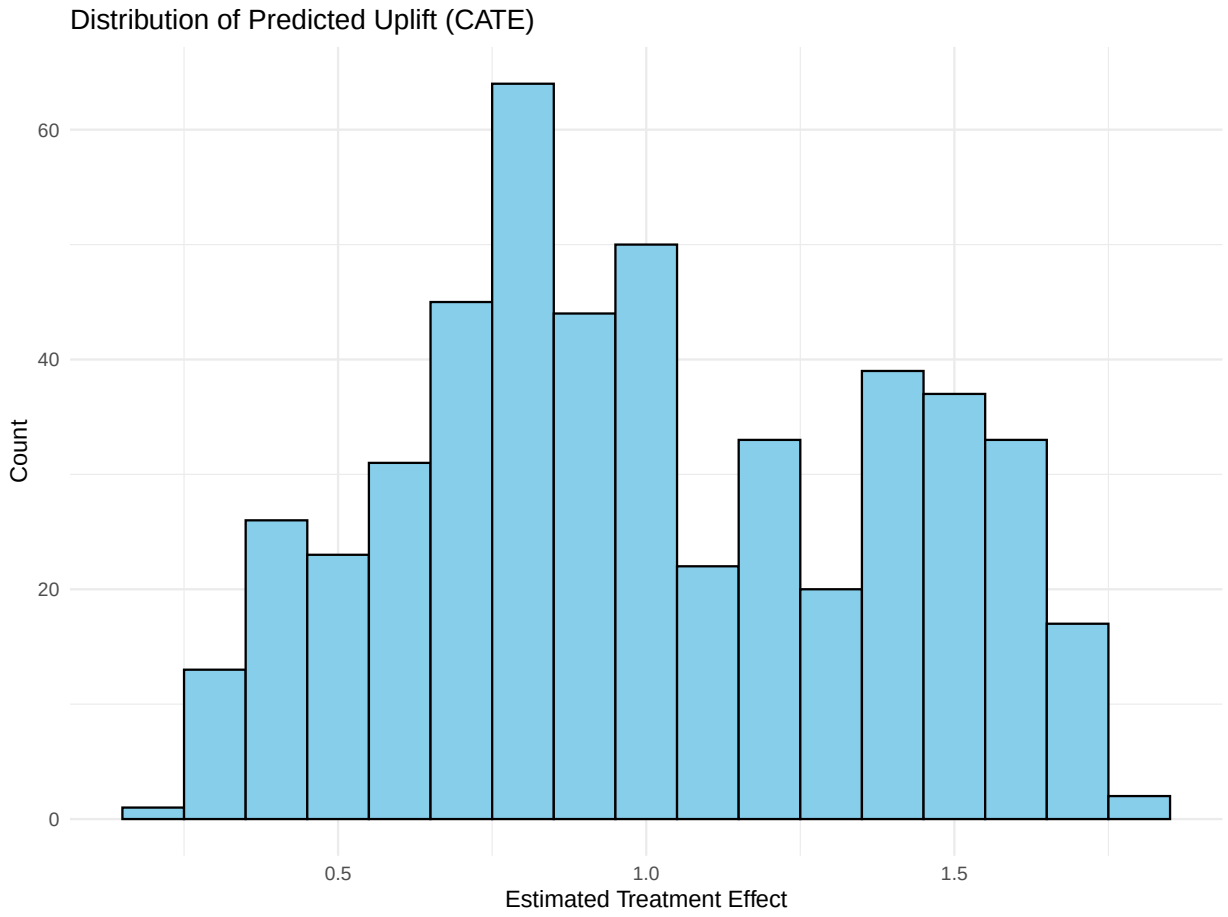
132 `[1] 0.7134650 0.8063268 1.7933776 1.2126532 1.0135543 1.4721875`

133 **Visualization**

```
library(ggplot2)

# Create dataframe for visualization
uplift_df <- data.frame(
  uplift = cate_pred$predictions
)
```



```
# Histogram of predicted uplift scores
ggplot(uplift_df, aes(x = uplift)) +
  geom_histogram(binwidth = 0.1, fill = "skyblue", color = "black") +
  labs(title = "Distribution of Predicted Uplift (CATE)",
       x = "Estimated Treatment Effect", y = "Count") +
  theme_minimal()
```



Association Rules and Collaborative Filtering (Ch.14)

Association Rules (Market Basket Analysis)

Data


```r
##install.packages("arulesViz")
#install.packages("arules")
library(arules)
library(arulesViz)
# Simulate simple transaction list
transactions_list <- list(
  c("milk", "bread"),
  c("bread", "butter"),
  c("milk", "eggs"),
  c("milk", "bread", "butter"),
  c("bread", "eggs"),
  c("milk", "bread"),
  c("butter", "eggs")
)
```

Generate association rules

```r
# Convert to transaction format
trans <- as(transactions_list, "transactions")

rules <- apriori(trans, parameter = list(supp = 0.3, conf = 0.6))
```

```
Apriori

Parameter specification:
 confidence minval smax arem  aval originalSupport maxtime support minlen
        0.6    0.1    1 none FALSE            TRUE       5     0.3      1
 maxlen target  ext
     10  rules TRUE

Algorithmic control:
 filter tree heap memopt load sort verbose
    0.1 TRUE TRUE  FALSE TRUE    2    TRUE

Absolute minimum support count: 2

set item appearances ...[0 item(s)] done [0.00s].
set transactions ...[4 item(s), 7 transaction(s)] done [0.00s].
sorting and recoding items ... [4 item(s)] done [0.00s].
creating transaction tree ... done [0.00s].
checking subsets of size 1 2 done [0.00s].
writing ... [3 rule(s)] done [0.00s].
creating S4 object  ... done [0.00s].
```



```
arules::inspect(head(rules, n = 5, by = "lift"))
```

```
    lhs        rhs     support   confidence coverage  lift count
[1] {milk}  => {bread} 0.4285714 0.7500000  0.5714286 1.05 3
[2] {bread} => {milk}  0.4285714 0.6000000  0.7142857 1.05 3
[3] {}      => {bread} 0.7142857 0.7142857  1.0000000 1.00 5
```

Visualization

```
# install.packages("arulesViz")
library(arulesViz)

plot(rules, method = "graph", engine = "htmlwidget")
```

Collaborative Filtering (User-Item Recommendation)

Data

```
# install.packages("recommenderlab")
library(recommenderlab)
ratings <- matrix(c(
  5, NA, 3, 1,   # User 1
  4, 2, NA, 1,   # User 2
  NA, 5, 4, 2,   # User 3
  1, 2, 3, NA    # User 4
), nrow = 4, byrow = TRUE)

colnames(ratings) <- c("Item_A", "Item_B", "Item_C", "Item_D")
rownames(ratings) <- paste0("User_", 1:4)

rrm <- as(ratings, "realRatingMatrix")
```

Train Collaborative Filtering Model

```
model <- Recommender(rrm, method = "UBCF")
```


Top2 Recommendations for User 1

```
prediction1 <- predict(model, rrm[1], n = 2)
as(prediction1, "list")
```

```
$`0`
[1] "Item_B"
```

```
prediction2 <- predict(model, rrm[2], n = 2)
as(prediction2, "list")
```

```
$`0`
[1] "Item_C"
```

```
prediction3 <- predict(model, rrm[3], n = 2)
as(prediction3, "list")
```

```
$`0`
[1] "Item_A"
```

```
prediction4 <- predict(model, rrm[4], n = 2)
as(prediction4, "list")
```

```
$`0`
[1] "Item_D"
```

Cluster Analysis (Ch.15)

Data

```
# install.packages("ggplot2")
# install.packages("factoextra")
library(ggplot2)
library(factoextra)
# Simulated data for 3 groups
set.seed(123)
group1 <- data.frame(X = rnorm(50, mean = 2), Y = rnorm(50, mean = 2))
group2 <- data.frame(X = rnorm(50, mean = 6), Y = rnorm(50, mean = 6))
```



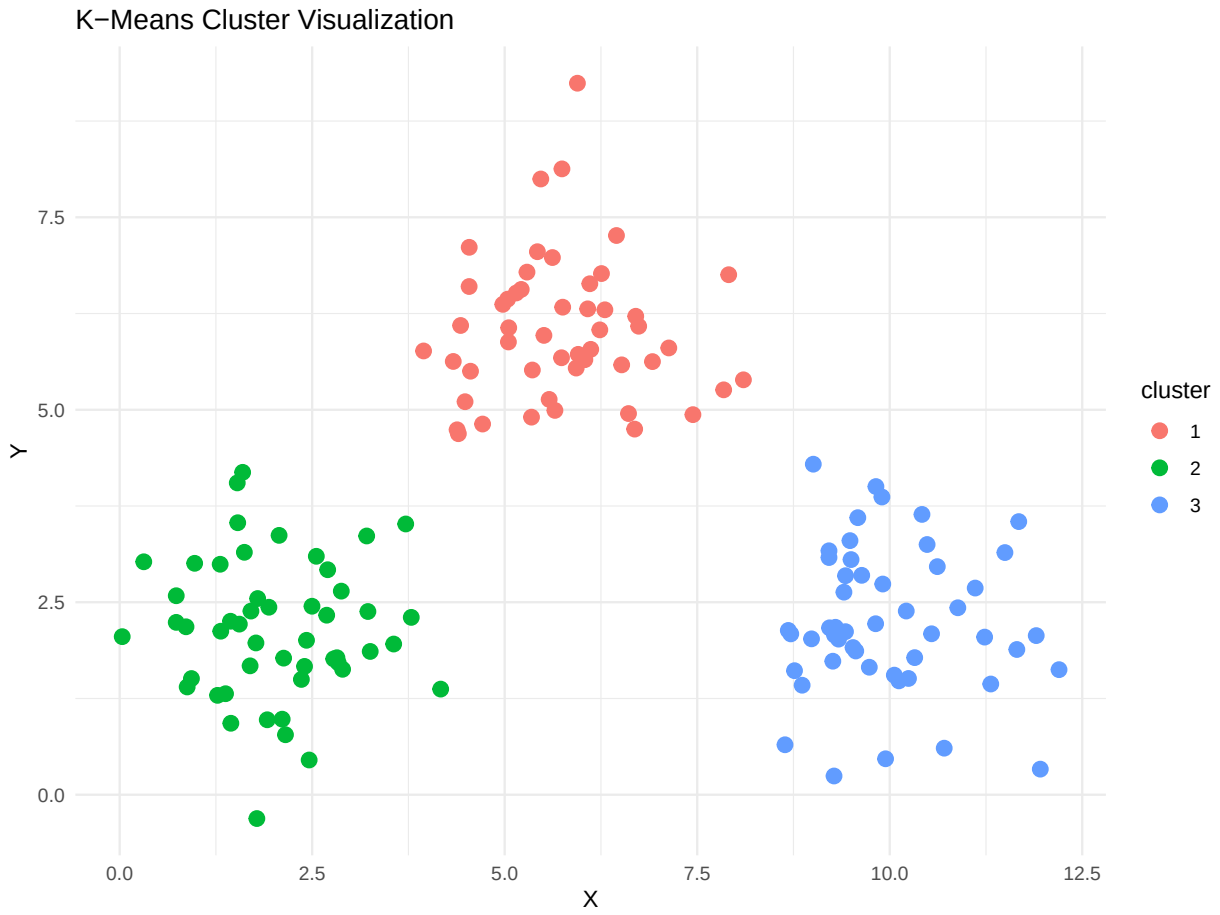
```
group3 <- data.frame(X = rnorm(50, mean = 10), Y = rnorm(50, mean = 2))

data <- rbind(group1, group2, group3)
```

179 **Clustering with k-means**

```
set.seed(123)
kmeans_result <- kmeans(data, centers = 3, nstart = 25)

# Append cluster labels
data$cluster <- as.factor(kmeans_result$cluster)
```

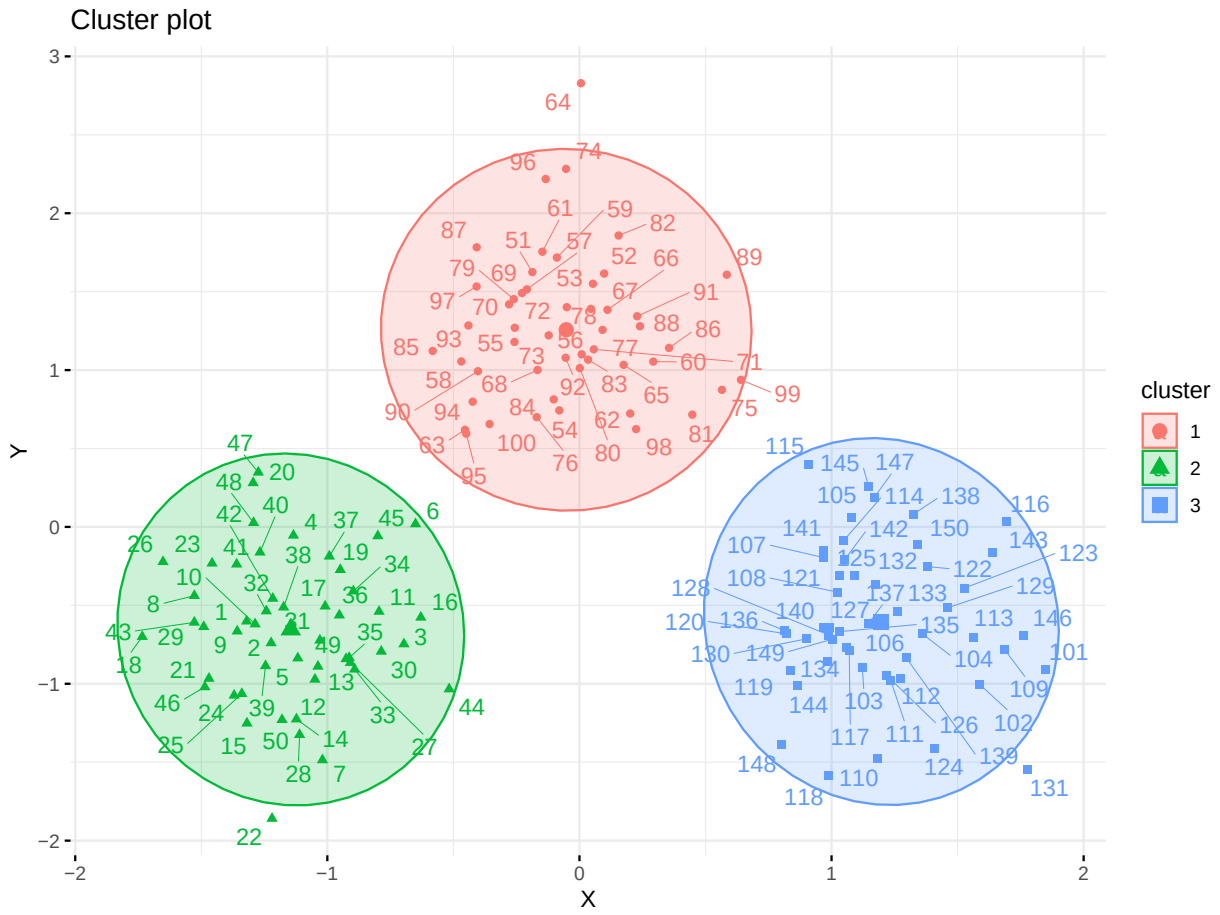
180 **Visualization using ggplot2 and factoextra**

```
# Simple scatter plot
ggplot(data, aes(x = X, y = Y, color = cluster)) +
  geom_point(size = 3) +
  labs(title = "K-Means Cluster Visualization") +
  theme_minimal()
```

181

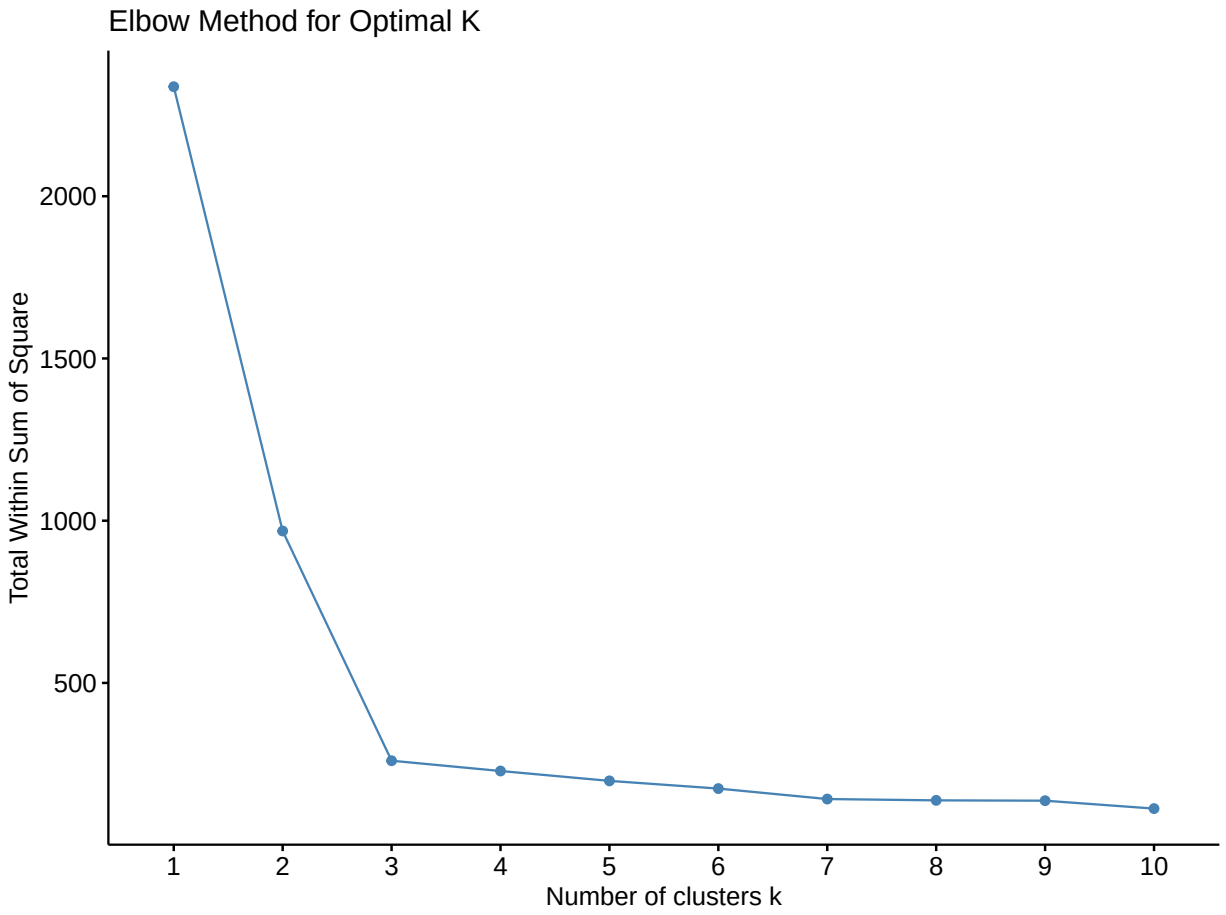
```
fviz_cluster(kmeans_result, data = data[, 1:2],
             ellipse.type = "norm", repel = TRUE,
             ggtheme = theme_minimal())
```

182

183 **Evaluation using cluster plots**

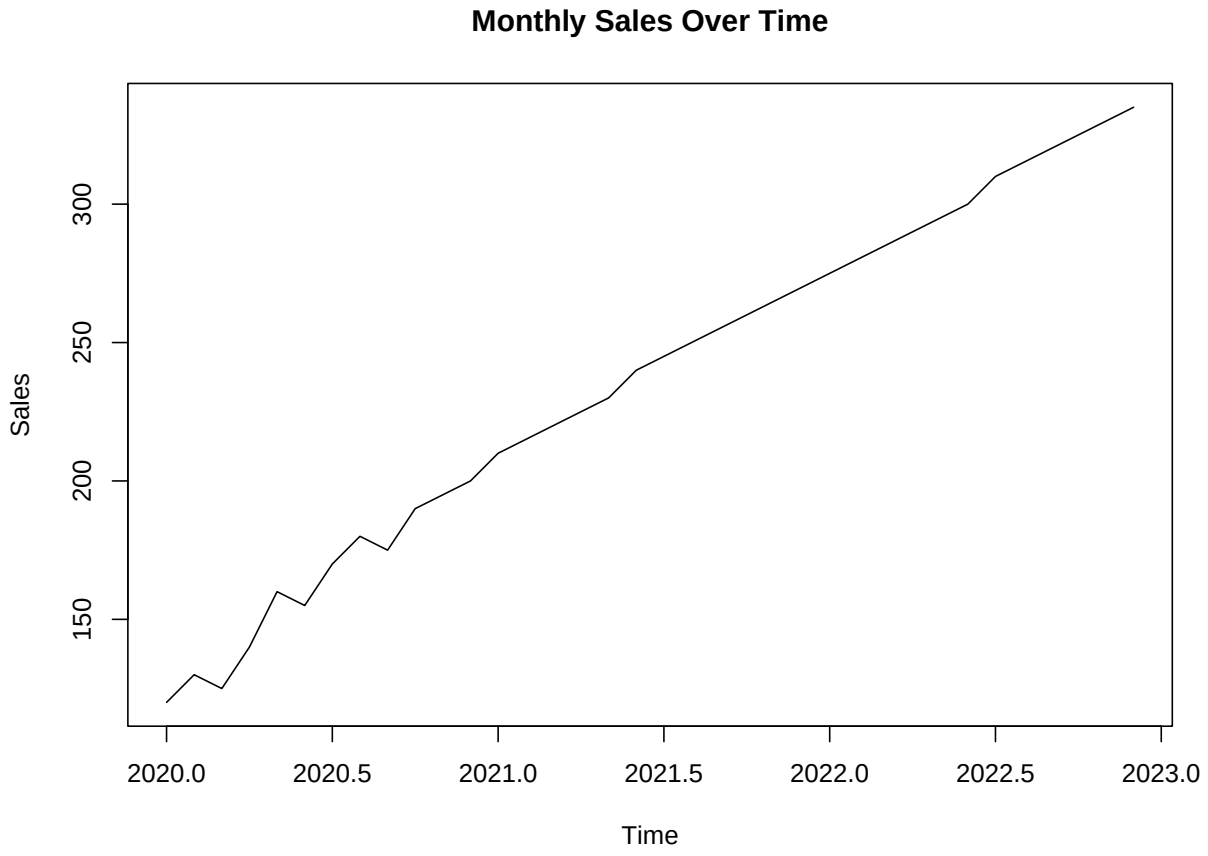
```
fviz_nbclust(data[, 1:2], kmeans, method = "wss") +
  labs(title = "Elbow Method for Optimal K")
```

Handling Time Series (Ch.16)

Data

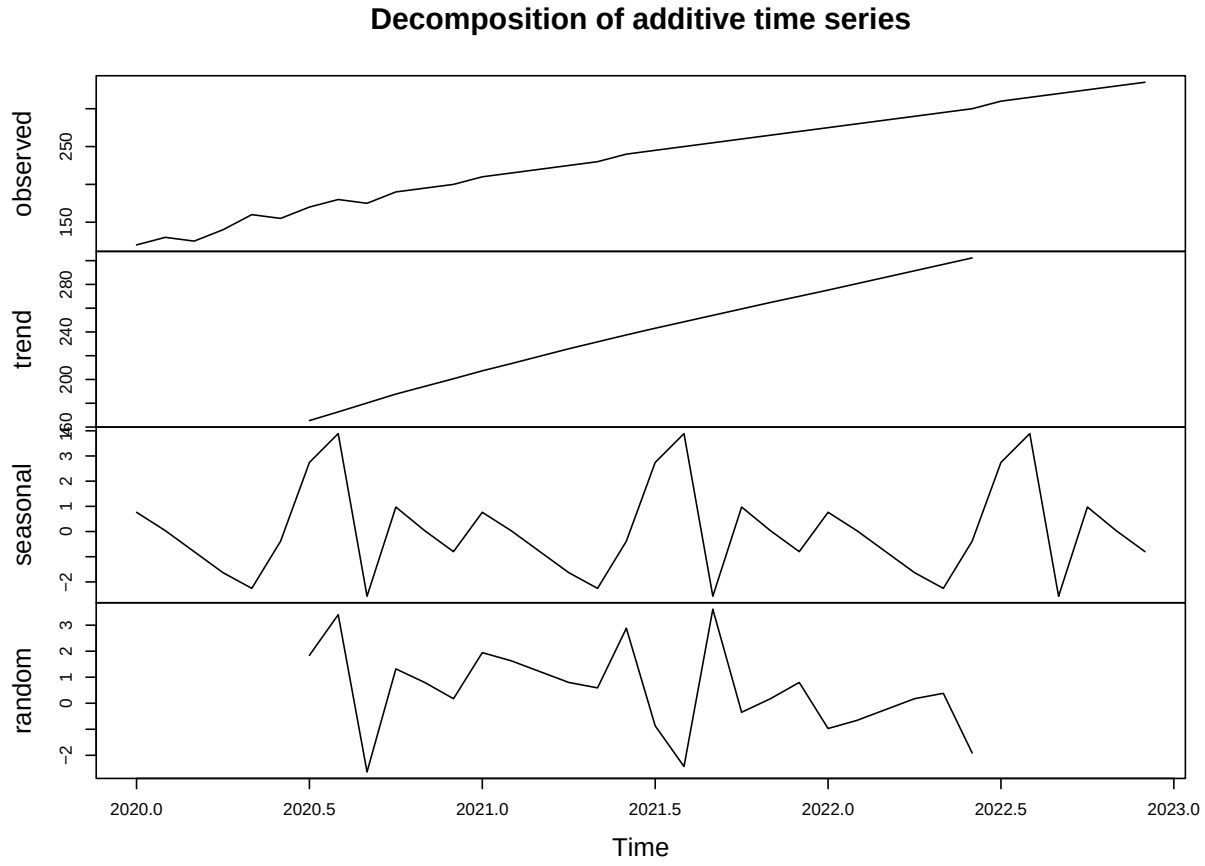
```
# install.packages("forecast")
# install.packages("tseries")
library(forecast)
library(tseries)
# Monthly sales data over 3 years
sales <- c(120, 130, 125, 140, 160, 155, 170, 180, 175, 190, 195, 200,
           210, 215, 220, 225, 230, 240, 245, 250, 255, 260, 265, 270,
           275, 280, 285, 290, 295, 300, 310, 315, 320, 325, 330, 335)

ts_data <- ts(sales, start = c(2020, 1), frequency = 12)
plot(ts_data, main = "Monthly Sales Over Time", ylab = "Sales", xlab = "Time")
```

187

188 **Decomposition into trend, seasonality, and residuals**

```
decomp <- stats::decompose(ts_data)
plot(decomp)
```

Stationarity testing

```
adf.test(ts_data)  # Augmented Dickey-Fuller Test
```

```
    Augmented Dickey-Fuller Test

data:  ts_data
Dickey-Fuller = -5.5852, Lag order = 3, p-value = 0.01
alternative hypothesis: stationary
```

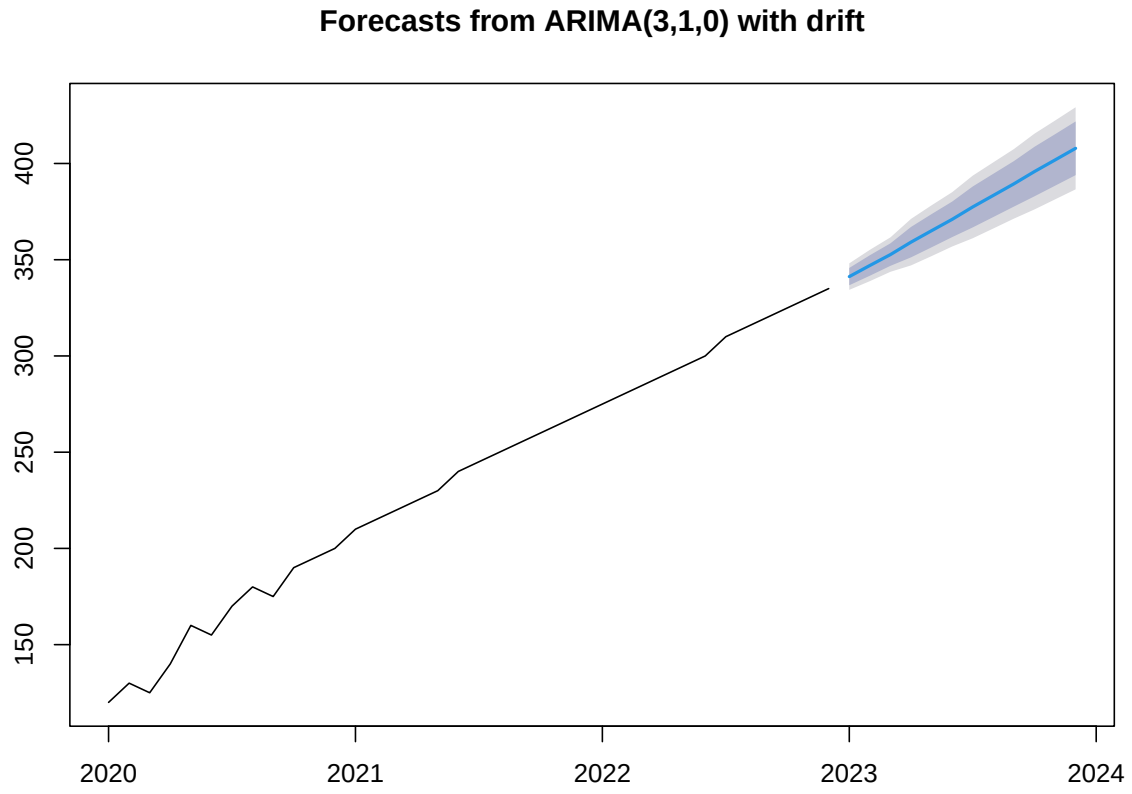
Forecasting using ARIMA


```r
model <- auto.arima(ts_data)
summary(model)
```

```
Series: ts_data
ARIMA(3,1,0) with drift

Coefficients:
          ar1      ar2     ar3   drift
      -0.3691  -0.2281  0.5243  6.1481
s.e.   0.1426   0.1514  0.1459  0.5115

sigma^2 = 12.32:  log likelihood = -92.47
AIC=194.95   AICc=197.02   BIC=202.72

Training set error measures:
                      ME     RMSE      MAE        MPE     MAPE      MASE
Training set -0.05025393 3.256568 2.264695 0.07513985 1.160768 0.0316004
                    ACF1
Training set 0.05778946
```

Visualization with forecast and ggplot2

```r
forecast_values <- forecast(model, h = 12)  # forecast next 12 months
plot(forecast_values)
```

Regression-Based Forecasting (Ch.17)

Data

```
# install.packages("ggplot2")
# install.packages("forecast")
library(ggplot2)
library(forecast)
# Monthly sales with a linear upward trend and slight seasonality
months <- 1:36
sales <- 100 + 2 * months + 10 * sin(2 * pi * months / 12) + rnorm(36, 0, 5)
data <- data.frame(Month = months, Sales = sales)
```


Fit Time-based Regression Model

```r
# Include a seasonal dummy variable (month as factor)
data$Season <- as.factor((data$Month - 1) %% 12 + 1)

# Fit regression model with time and seasonality
reg_model <- lm(Sales ~ Month + Season, data = data)
summary(reg_model)
```

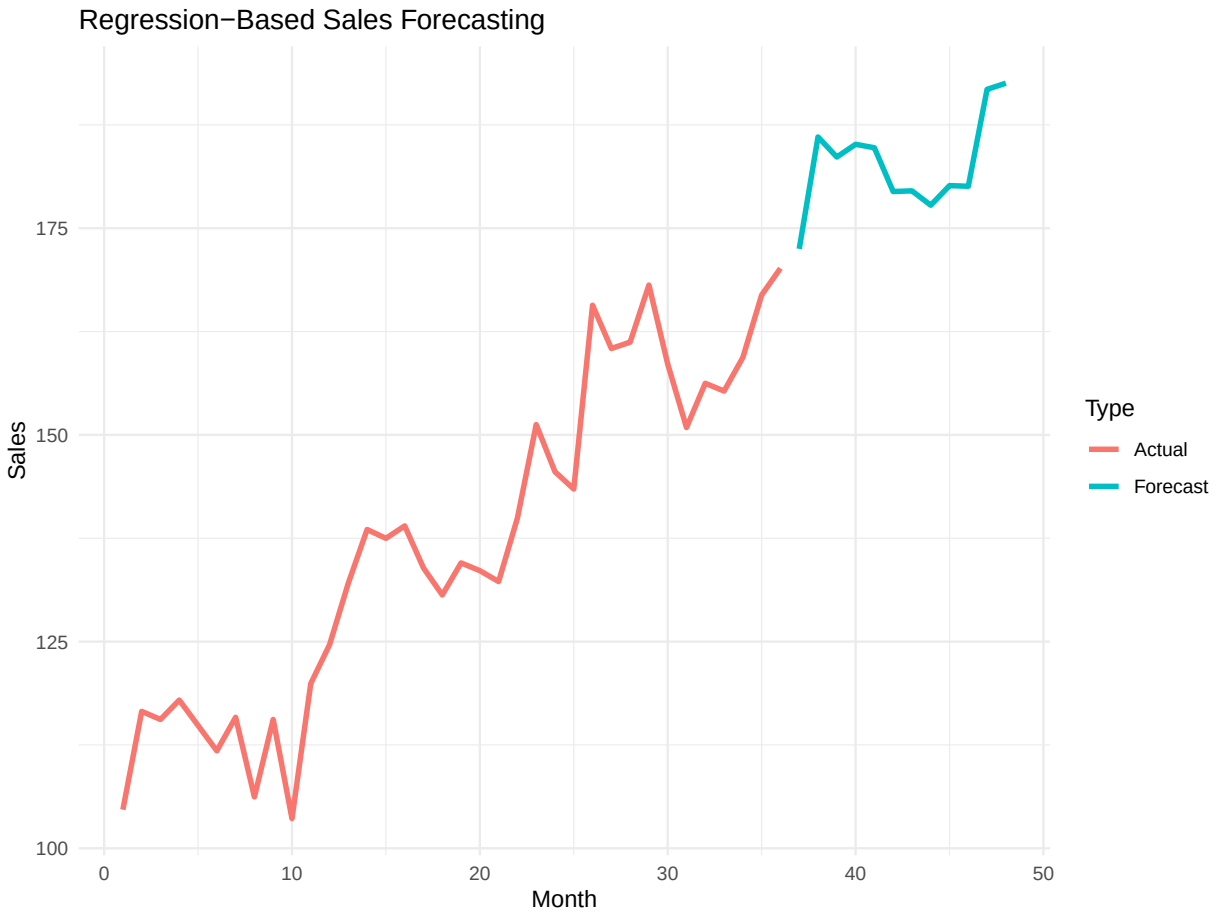
```

Call:
lm(formula = Sales ~ Month + Season, data = data)

Residuals:
    Min      1Q  Median      3Q     Max 
-7.8235 -1.9715  0.1086  1.6818  6.2781 

Coefficients:
             Estimate Std. Error t value Pr(>|t|)    
(Intercept) 101.94068    2.57741  39.552  < 2e-16 ***
Month         1.90714    0.07091  26.894  < 2e-16 ***
Season2      11.62849    3.40461   3.416  0.00237 ** 
Season3       7.29403    3.40682   2.141  0.04310 *  
Season4       6.92547    3.41051   2.031  0.05401 .  
Season5       4.60199    3.41567   1.347  0.19100    
Season6      -2.60163    3.42229  -0.760  0.45486    
Season7      -4.42403    3.43036  -1.290  0.20998    
Season8      -8.07087    3.43988  -2.346  0.02795 *  
Season9      -7.60636    3.45082  -2.204  0.03779 *  
Season10     -9.59260    3.46319  -2.770  0.01090 *  
Season11      0.22835    3.47695   0.066  0.94820    
Season12     -0.94182    3.49211  -0.270  0.78980    
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 4.169 on 23 degrees of freedom
Multiple R-squared:  0.9716,    Adjusted R-squared:  0.9567 
F-statistic: 65.51 on 12 and 23 DF,  p-value: 7.543e-15
```

Forecast Futrue Sales (Next 12 Months)


```r
future <- data.frame(
  Month = 37:48,
  Season = as.factor((37:48 - 1) %% 12 + 1)
)

future$Prediction <- predict(reg_model, newdata = future)
```

Visualize Actual + Forecated values

```r
# Combine for plotting
combined <- rbind(
  data.frame(Month = data$Month, Sales = data$Sales, Type = "Actual"),
  data.frame(Month = future$Month, Sales = future$Prediction, Type = "Forecast")
)

ggplot(combined, aes(x = Month, y = Sales, color = Type)) +
  geom_line(size = 1.2) +
  labs(title = "Regression-Based Sales Forecasting",
       x = "Month", y = "Sales") +
  theme_minimal()
```

Smoothing Methods (Ch.18)

Data

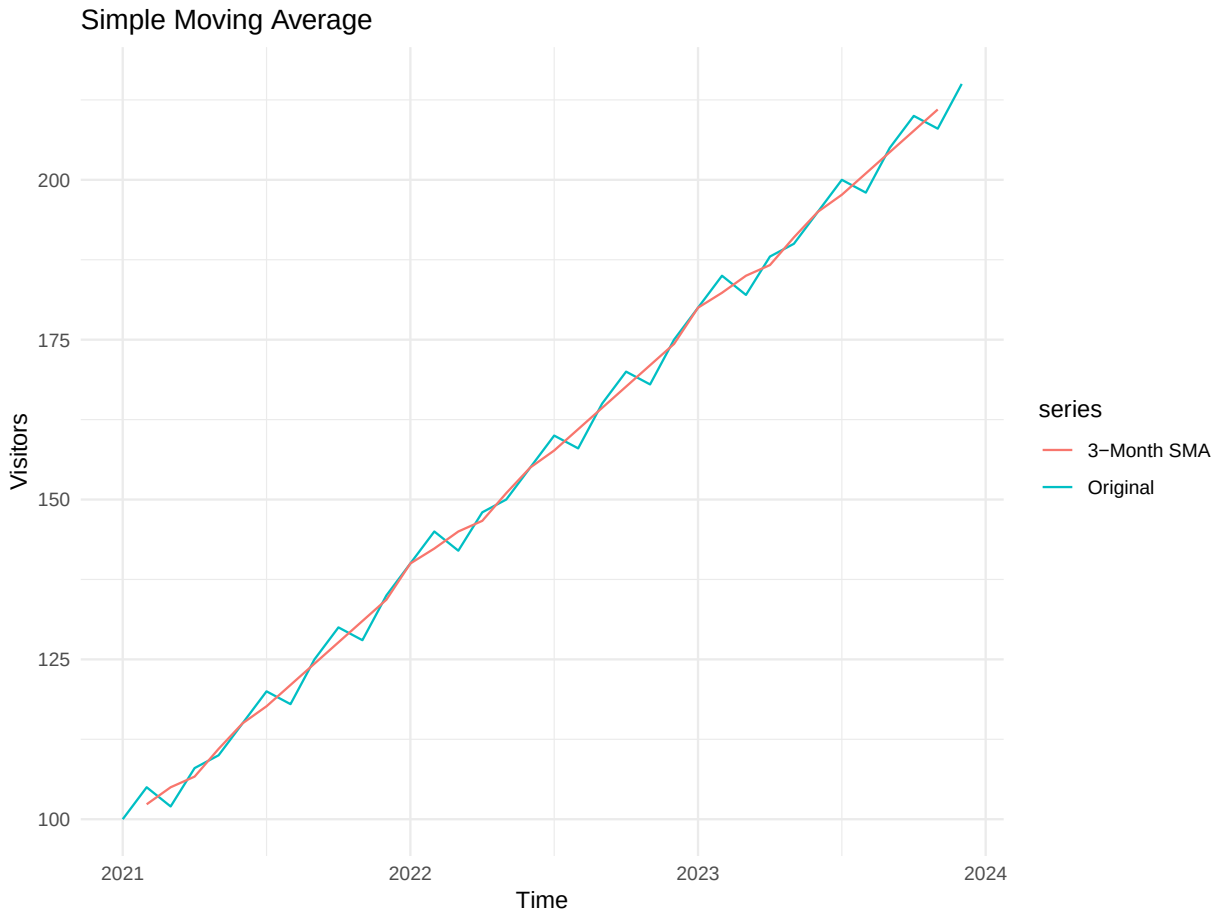
```
# install.packages("forecast")
# install.packages("ggplot2")
library(forecast)
library(ggplot2)
# Monthly visitor data
# Extended visitor data (3 years of monthly values)
visitors <- c(
  100, 105, 102, 108, 110, 115, 120, 118, 125, 130, 128, 135,
  140, 145, 142, 148, 150, 155, 160, 158, 165, 170, 168, 175,
  180, 185, 182, 188, 190, 195, 200, 198, 205, 210, 208, 215
)
```



```
ts_data <- ts(visitors, start = c(2021, 1), frequency = 12)
```

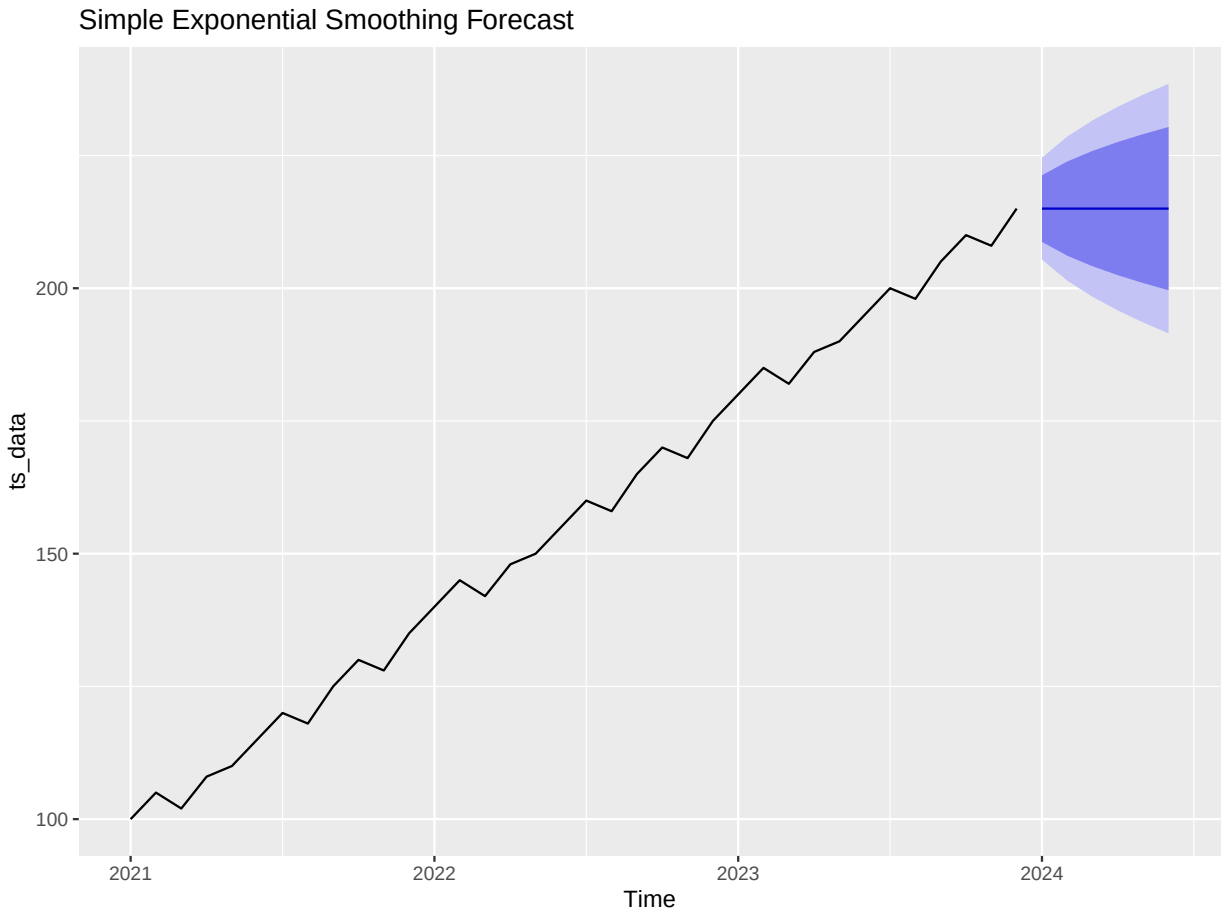
Apply simple moving average (SMA)

```
# SMA with 3-month window
sma_values <- ma(ts_data, order = 3)

# Plot original + SMA
autoplot(ts_data, series = "Original") +
  autolayer(sma_values, series = "3-Month SMA") +
  labs(title = "Simple Moving Average", y = "Visitors", x = "Time") +
  theme_minimal()
```



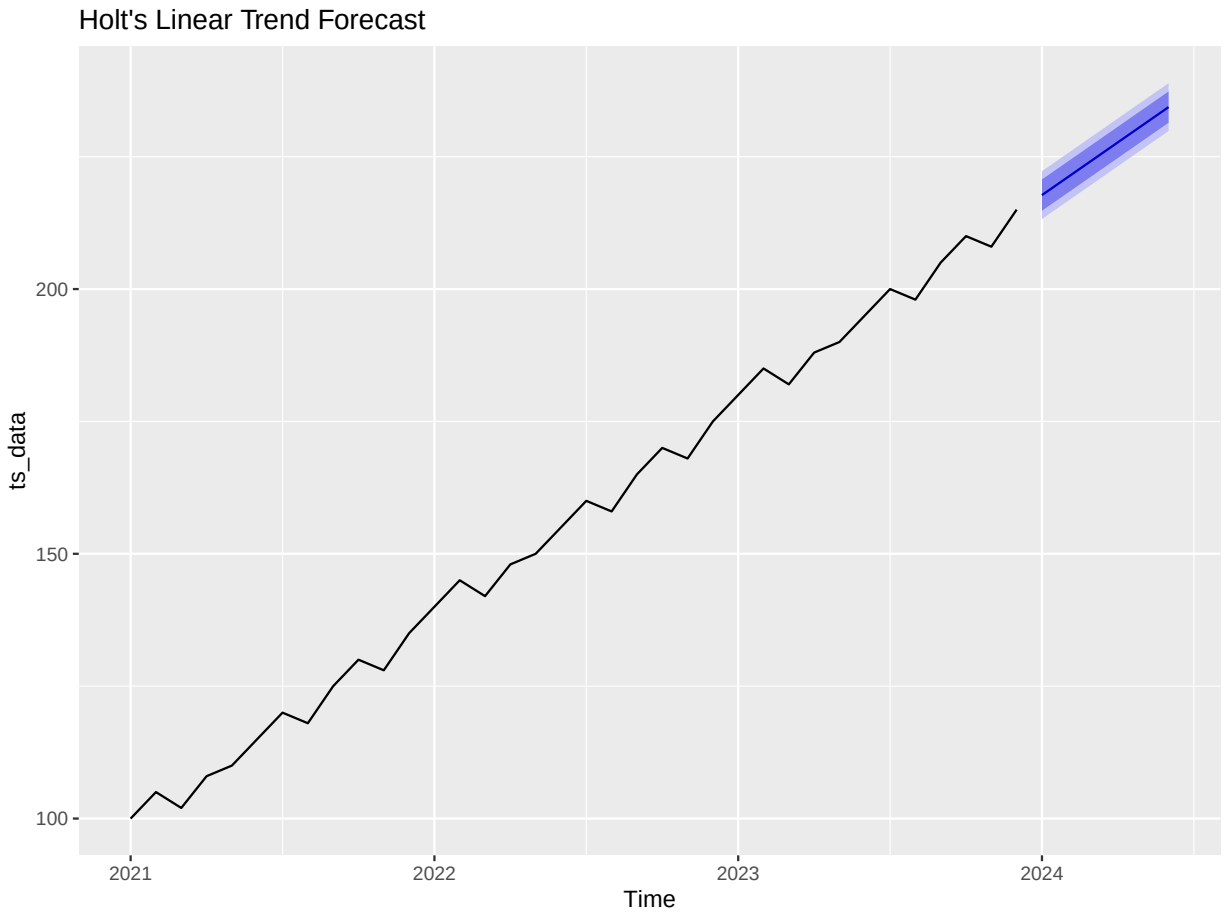
Apply9ing simple exponential smoothing (SES)

```
ses_model <- ses(ts_data, h = 6)
autoplot(ses_model) +
  labs(title = "Simple Exponential Smoothing Forecast")
```



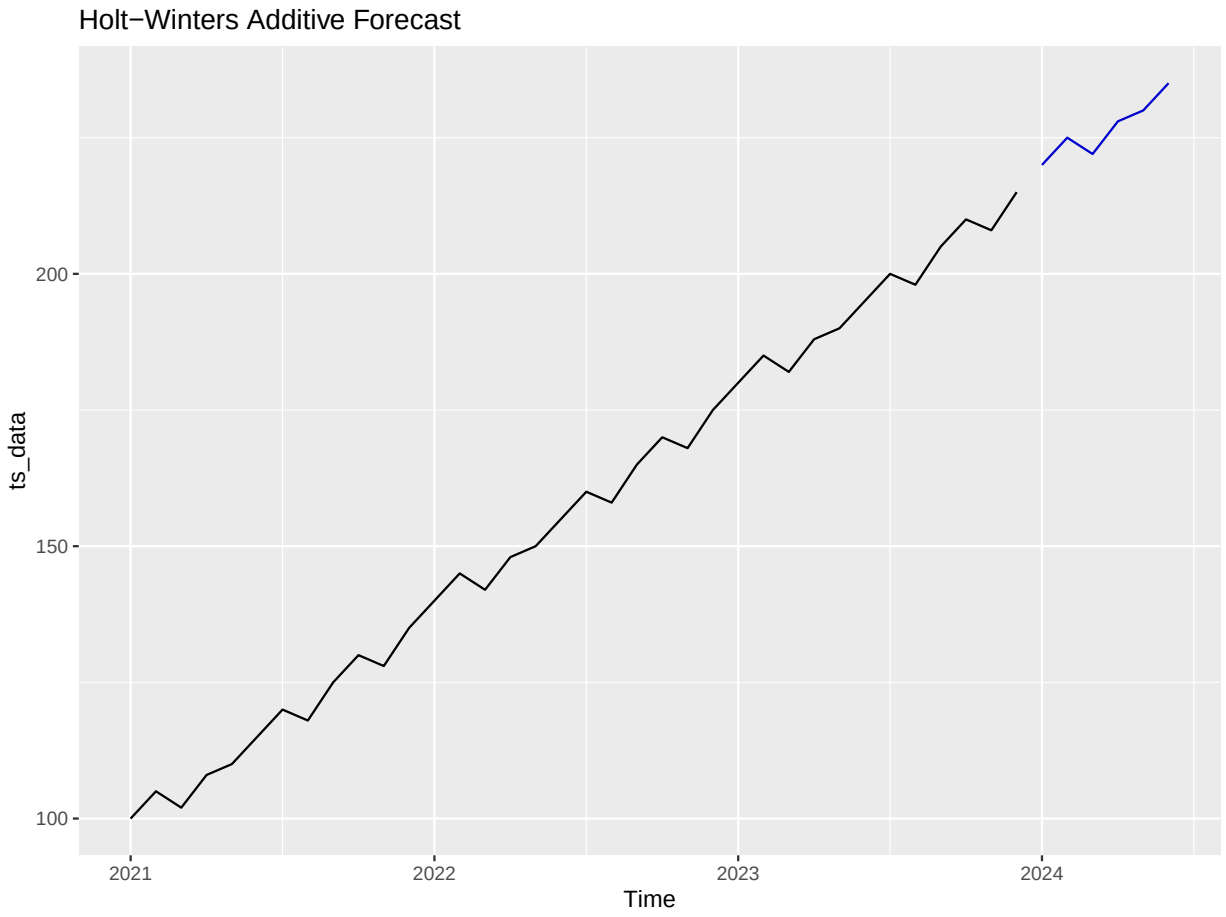
Holt's Linear Trend Method

```
holt_model <- holt(ts_data, h = 6)
autoplot(holt_model) +
  labs(title = "Holt's Linear Trend Forecast")
```

Holt-Winters (Seasonal Smoothing)

```
hw_model <- hw(ts_data, h = 6, seasonal = "additive")
autoplot(hw_model) +
  labs(title = "Holt-Winters Additive Forecast")
```

Social Network Analytics (Ch.19)

Data

```
# install.packages("igraph")
library(igraph)
# Sample edge list (directed network)
edges <- data.frame(
  from = c("A", "A", "B", "C", "D", "E", "F", "G", "G", "H"),
  to   = c("B", "C", "C", "D", "E", "F", "G", "H", "A", "F")
)
```

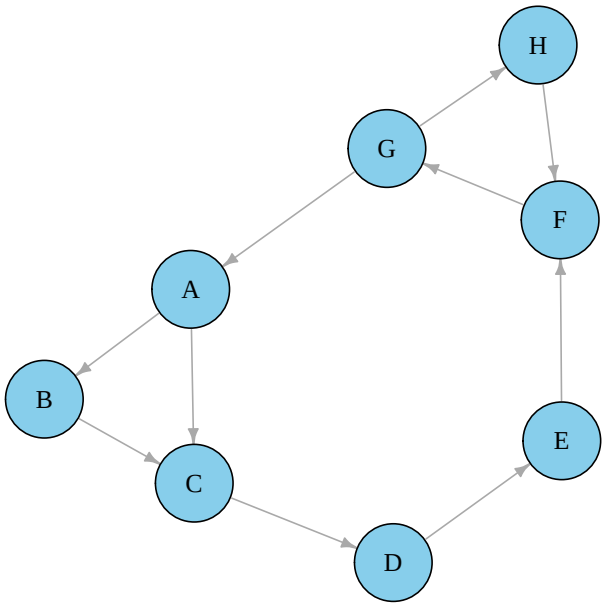

Create graph object

```
g <- graph_from_data_frame(edges, directed = TRUE)
g
```

```
IGRAPH 5be3d25 DN-- 8 10 --
+ attr: name (v/c)
+ edges from 5be3d25 (vertex names):
 [1] A->B A->C B->C C->D D->E E->F F->G G->H G->A H->F
```

Plot the graph

```
plot(g,
     vertex.color = "skyblue",
     vertex.size = 30,
     vertex.label.color = "black",
     edge.arrow.size = 0.5,
     main = "Directed Social Network Graph")
```

Analyze key metrics (centrality, degree, components)

```
# Degree Centrality
deg <- degree(g, mode = "all")
cat("Degree Centrality:\n")
```

```
Degree Centrality:
```

```
print(deg)
```

```
A B C D E F G H
3 2 3 2 2 3 3 2
```



```r
# Betweenness Centrality
btw <- betweenness(g)
cat("\nBetweenness Centrality:\n")
```

```

Betweenness Centrality:
```

```r
print(btw)
```

```
 A  B  C  D  E  F  G  H
18  0 18 18 18 26 26  1
```

```r
# Closeness Centrality
cls <- closeness(g)
cat("\nCloseness Centrality:\n")
```

```

Closeness Centrality:
```

```r
print(cls)
```

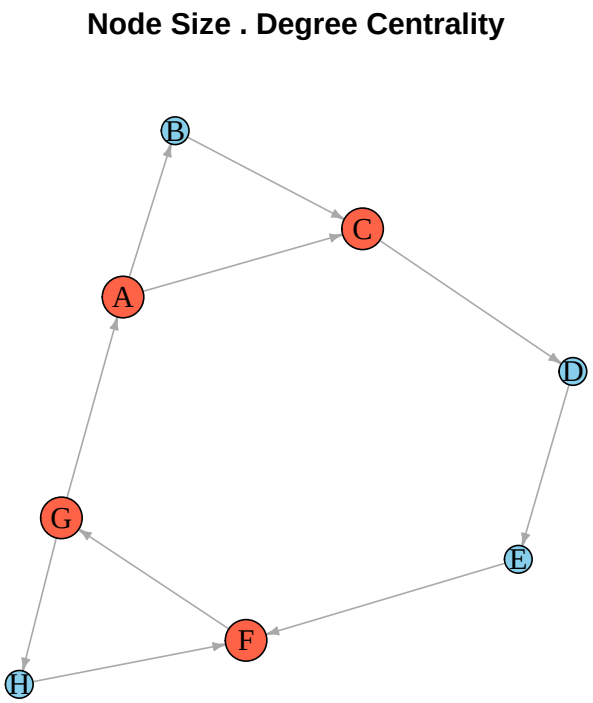
```
         A          B          C          D          E          F          G
0.04545455 0.03703704 0.03846154 0.04166667 0.04545455 0.05000000 0.06666667
         H
0.04000000
```

```r
# Components
components(g)
```

```
$membership
A B C D E F G H
1 1 1 1 1 1 1 1

$csize
[1] 8

$no
[1] 1
```

Visualize node importance


```
V(g)$size <- deg * 5  # scale node size by degree
V(g)$color <- ifelse(deg == max(deg), "tomato", "skyblue")

plot(g,
     vertex.label.cex = 1.2,
     vertex.label.color = "black",
     edge.arrow.size = 0.4,
     main = "Node Size  Degree Centrality")
```



294

295 Text Mining (Ch.20)

296 Data


```
# install.packages("tm")
# install.packages("wordcloud")
# install.packages("RColorBrewer")
library(tm)
library(wordcloud)
library(RColorBrewer)
texts <- c(
  "Data mining is fun and useful.",
  "Machine learning is powerful and flexible.",
  "Text mining extracts meaning from documents.",
  "Natural language processing is a branch of AI.",
  "Data science combines statistics and computing."
)

corpus <- Corpus(VectorSource(texts))
```

297 **Text preprocessing (cleaning, tokenizing)**

```
corpus_clean <- tm_map(corpus, content_transformer(tolower))
corpus_clean <- tm_map(corpus_clean, removePunctuation)
corpus_clean <- tm_map(corpus_clean, removeNumbers)
corpus_clean <- tm_map(corpus_clean, removeWords, stopwords("english"))
corpus_clean <- tm_map(corpus_clean, stripWhitespace)
```

298 **Document-Term Matrix creation**

```
dtm <- DocumentTermMatrix(corpus_clean)
matrix <- as.matrix(dtm)
word_freq <- sort(colSums(matrix), decreasing = TRUE)
head(word_freq)
```

```
299     data   mining      fun   useful flexible learning
300        2        2        1        1        1        1
```

301 **Frequency analysis**

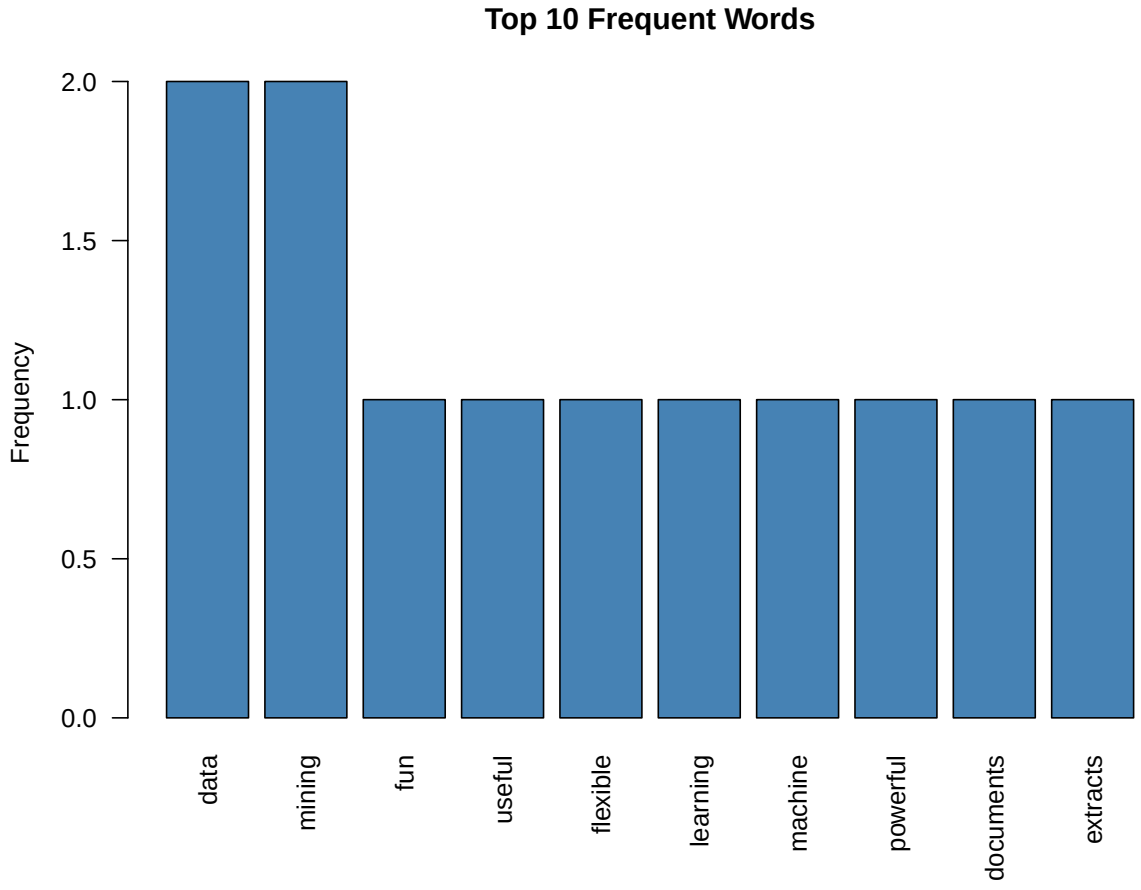
```
word_freq
```



```
      data     mining        fun     useful   flexible   learning    machine
         2          2          1          1          1          1          1
  powerful  documents   extracts    meaning       text     branch   language
         1          1          1          1          1          1          1
   natural processing   combines  computing    science statistics
         1          1          1          1          1          1
```

Visualization (Word cloud and bar plot)

```
barplot(word_freq[1:10],
        las = 2,
        col = "steelblue",
        main = "Top 10 Frequent Words",
        ylab = "Frequency")
```




```r
set.seed(123)
wordcloud(
  words = names(word_freq),
  freq = word_freq,
  min.freq = 1,
  scale = c(4, 0.8),
  colors = brewer.pal(8, "Dark2")
)
```

310

